CT1 Answer Sheet — NLP (21CSE583T)

Exam-ready | Examples from your notes only | No new memorization

> Answer **any TWO** questions (2 × 20 = 40 marks)

Question 1 — Social Media Sentiment Analysis

(a) Text Preprocessing in NLP and Sentiment Analysis [10 marks]

Definition: Text preprocessing transforms raw, noisy text into a clean, standardized format suitable for machine learning. It is the first and most critical step in any NLP pipeline.

Key Preprocessing Steps *(from Unit 1 notes)*:

1. **Lowercasing** — Converts all text to lowercase for uniformity
   - "HAPPY" → "happy"
2. **Removing URLs & Mentions** — Strips social media noise
   - "@user check https://example.com" → "check"
3. **Removing Special Characters & Punctuation** — Eliminates noise symbols
   - "Great!!! #amazing" → "Great amazing"
4. **Handling Emojis** — Convert or remove emojis
   - "Love this 😊 " → "Love this [happy_emoji]" or "Love this"
5. **Stop Word Removal** — Removes common words with little sentiment value
   - "This is a great product" → "great product"
   - ⚠ Note: "not" in "not happy" carries meaning — don't blindly remove all stop words
6. **Stemming / Lemmatization** — Reduces words to root form
   - Stemming (fast, crude): "running" → "run", but "caring" → "car" (error-prone)
   - Lemmatization (accurate, context-aware): "better" → "good", "am/are/is" → "be"
7. **Handling Abbreviations & Slang** — Normalizes informal text
   - "u r gr8" → "you are great"

Why Preprocessing is Crucial for Sentiment Analysis:

Reason	Impact
Noise Reduction	Removes hashtags, URLs, mentions that mislead classifiers
Standardization	"LOVE", "love", "Love" treated as one word

Reason	Impact
Better Features	Clean text → more accurate ML features → higher model accuracy
Computational Efficiency	Smaller vocabulary → faster training, lower memory

Example Workflow:

```
Raw:    "@John OMG!!! This is AMAZING 😍 https://bit.ly/xyz"
Steps:  Remove URL → Remove mention → Remove !!! → Lowercase → Remove stop
words
Clean:  "amazing"
Sentiment: Positive (high confidence)
```

Conclusion: Without preprocessing, models are confused by noise, inconsistency, and social media informality. Proper preprocessing can improve accuracy by 15–30%.

(b) Tokenization — Significance and Methods for Social Media [10 marks]

Definition: Tokenization is breaking text into smaller units called **tokens** (words, subwords, or characters). It is the bridge between raw text and numerical representations that ML models can process.

Significance:

1. **Foundation for all NLP tasks** — every downstream task (classification, NER, sentiment) requires tokens
2. **Vocabulary creation** — determines what the model "knows"
3. **OOV (Out-of-Vocabulary) handling** — different methods handle unknown words differently; critical for social media where new slang appears constantly

Comparison of Tokenization Methods for Social Media:

Method	How it works	Social Media Elements	OOV Handling	Best For
Word-level	Split on whitespace/punctuation	❌ Poor (breaks hashtags)	❌ Cannot handle	Standard text only
Character-level	Split into individual characters	✅ Handles anything	✅ No OOV possible	Handles misspellings, but loses word meaning
Subword (BPE/WordPiece)	Merge frequent character pairs	✅ Good	✅ Breaks into known subwords	Best for rare words & hashtags

Method	How it works	Social Media Elements	OOV Handling	Best For
Tweet Tokenizer	Purpose-built for social media	✅ Excellent (preserves @, #, 😊)	❌ Still has OOV	Preserving social media semantics

Subword Example (BPE) with social media text:

```
Input:  "#COVID19 omgggg ur amazinggg"
Tokens: ["#", "COVID", "19", "om", "gg", "gg", "ur", "amazing", "gg"]
```

Handles: hashtags, repeated letters ("omgggg"), elongated words ("amazinggg")

Tweet Tokenizer Example:

```
Input:  "This is AMAZING!!! 😊  @user #happy"
Tokens: ["This", "is", "AMAZING", "!!!", "😊 ", "@user", "#happy"]
```

Preserves: emojis, hashtags, mentions, repeated punctuation

Recommendation: For social media sentiment analysis, use a **Hybrid approach**:

- Tweet Tokenizer first → preserves social media elements
- Subword tokenization second → handles OOV/slang/creative spellings

This combines the best of both: semantic preservation + OOV robustness.

Question 2 — Research Paper Recommendation System

(a) Importance of Text Vectorization in NLP and Recommendation Systems [10 marks]

Definition: Text vectorization converts text into numerical vectors that machine learning algorithms can process. Since computers cannot understand raw text, this is a mandatory transformation step.

Why Vectorization is Needed:

1. **ML Compatibility** — algorithms require numbers, not words
2. **Semantic Representation** — captures meaning and word relationships
3. **Enables Similarity Computation** — mathematical comparison between papers
4. **Scalability** — once vectorized, comparing millions of papers is fast

Role in Research Paper Recommendation:

1. Document Representation Each paper abstract → fixed-size vector capturing its semantic content

```
"Deep learning models for NLP..." → [0.23, 0.45, 0.12, ..., 0.78]
```

2. Similarity Computation (Cosine Similarity)

```
cos(θ) = (V1 · V2) / (‖V1‖ × ‖V2‖)

Paper A (deep learning): similarity with Paper B (deep learning) = 0.95
✅
Paper A (deep learning): similarity with Paper C (quantum computing) =
0.32  ❌
```

3. User Profile Creation Aggregate vectors of papers the user has read → user interest vector

```
User reads 3 ML papers → average their vectors → User Profile = [0.70,
0.30, 0.13]
```

4. Recommendation Generation Compare user profile against unread papers → rank by similarity → recommend top-k

System Pipeline:

```
Data Collection → Preprocessing → Vectorization → Store Vectors
→ User Profile → Cosine Similarity → Rank → Recommend → User Feedback
```

Key Benefits:

- **Cold start handling** — works for new users via content similarity
- **Semantic search** — find related papers even with different terminology
- **Topic clustering** — group papers by research area automatically

---

(b) Compare BoW, TF-IDF, and Word Embeddings [10 marks]

> All three examples use the **same corpus from your notes** — no new memorization needed.

Common Corpus *(from Unit 1 notes)*:

- Document #1: "He is a good boy. She is also good."
- Document #2: "Radhika is a good person."

Vocabulary: a, also, boy, good, He, is, person, She, Radhika

---

1. Bag of Words (BoW)

Counts word frequency per document. Ignores order and grammar.

Example *(from Unit 1 notes — puppy example)*:

```
"it is a puppy and it is extremely cute"
→ {it: 2, puppy: 1, and: 1, cute: 1, extremely: 1, ...}
```

For the He/Radhika corpus:

```
          [a, also, boy, good, He, is, person, She, Radhika]
Doc #1:  [1,   1,   1,   2,  1,  2,   0,     1,    0   ]
Doc #2:  [1,   0,   0,   1,  0,  1,   1,     0,    1   ]
```

Limitations: Ignores word order ("not good" = "good not"), no semantics, high-dimensional sparse vector.

---

2. TF-IDF (Term Frequency × Inverse Document Frequency)

Weights words by importance: frequent in one doc but rare across corpus → high score.

Formula *(from Unit 1 notes)*:

```
TF(word, doc)  = frequency of word in doc / total words in doc
IDF(word)      = log( total docs / docs containing word )
TF-IDF         = TF × IDF
```

Calculation with He/Radhika corpus *(you already know these values)*:

Word	TF (Doc #1)	IDF	TF-IDF (Doc #1)
He	1/9 = 0.11	log(2/1) = 0.301	0.033
good	2/9 = 0.22	log(2/2) = **0**	**0**

Key insight: "good" appears in BOTH docs → IDF = 0 → not useful for distinguishing them. "He" only in Doc #1 → IDF = 0.301 → useful discriminator.

Advantage over BoW: Automatically downweights common words, highlights distinctive ones.

---

3. Word Embeddings (Word2Vec / GloVe / FastText)

Represents words as **dense, low-dimensional vectors** where semantically similar words are close together.

Key analogy *(from Unit 2 notes)*:

```
king − man + woman ≈ queen
```

Architecture *(Word2Vec — from Unit 2 notes)*:

- Input: one-hot vector (vocab size = 10,000)
- Hidden: embedding layer (50 neurons) — this IS the word embedding
- Output: softmax prediction (discarded after training)

CBOW vs Skip-gram *(same "Sun rises in the east" example from your notes)*:

- **CBOW**: Given context words (The, Sun, in, the) → predict target (**rises**)
- **Skip-gram**: Given target (**rises**) → predict context words (The, Sun, in, the)

Variants:

- **FastText** — works at subword level: `awesome → awe, wes, eso, som, ome` (handles OOV)
- **GloVe** — uses global co-occurrence matrix (ice/steam example from Unit 2)

Comprehensive Comparison:

Aspect	Bag of Words	TF-IDF	Word Embeddings
Vector type	Sparse, high-dim	Sparse, high-dim	Dense, low-dim (50–300)
Captures semantics	❌ No	❌ No	✅ Yes
Handles synonyms	❌ No	❌ No	✅ Yes
OOV words	❌ Cannot	❌ Cannot	✅ FastText can
Training needed	No	No	Yes (or pre-trained)
Interpretability	High	High	Low
Best use case	Baseline, spam	Search, ranking	Semantic similarity

For Research Paper Recommendation: Word Embeddings are best — papers using different terminology but similar concepts will still match (e.g., "neural nets" ≈ "deep learning").

Optimal hybrid: `Score = α × TF-IDF_similarity + β × Embedding_similarity`

Question 3 — Book Review Analysis

(a) LDA (Latent Dirichlet Allocation) and Topic Discovery [10 marks]

Definition: LDA is a **probabilistic generative model** for topic modeling. It assumes:

- Each **document** is a mixture of topics
- Each **topic** is a distribution of words
- "Latent" = hidden (topics are discovered automatically, no labels needed)

Core Concept:

```
Every document  →  blend of K topics   (e.g. 60% Topic A, 40% Topic B)
Every topic     →  distribution over words  (e.g. "broccoli" 30%, "banana"
15%)
```

LDA works **backwards** — given documents, find the topics most likely to have generated them.

Example *(from Unit 2 notes — same 5 documents)*:

Doc	Text
D1	"I like to eat broccoli and bananas."
D2	"I ate a banana and salad for breakfast."
D3	"Puppies and kittens are cute."
D4	"My sister adopted a kitten yesterday."
D5	"Look at this cute hamster munching on a piece of broccoli."

LDA with K = 2 topics discovers:

Topic	Top Words	Human Label
Topic A	broccoli 30%, bananas 15%, breakfast 10%, munching 10%	🥦 Food
Topic B	puppies 20%, kittens 20%, cute 20%, hamster 15%	🐾 Animals

Document-topic mixtures:

Doc	Topic A (Food)	Topic B (Animals)	Dominant
D1	100%	0%	Food
D2	100%	0%	Food
D3	0%	100%	Animals
D4	0%	100%	Animals
D5	**60%**	**40%**	Mixed

> LDA does **not** name topics — humans look at top words and assign labels.

How LDA Works (Gibbs Sampling — from Unit 2 notes):

```
1. Randomly assign every word in every document to a topic
2. Repeat many iterations:
   For each word w in document d:
```

```
      - Pretend w has no topic assignment
      - Ask: given the topic mix of d + how common w is in each topic → reassign w
3. After convergence: stable topic-word and document-topic distributions
```

Key Hyperparameters *(from Unit 2 notes)*:

Parameter	Meaning	Effect
K	Number of topics (must be preset)	Try 5–50 for typical corpora
α (alpha)	Document-topic concentration	Low → focused on few topics
β (beta)	Topic-word concentration	Low → narrow specific topics

---

How LDA Helps in Topic Discovery for Book Reviews:

1. **Automatic Genre Classification** — discovers Mystery, Romance, Fantasy, Sci-Fi without labels
2. **Multi-Genre Detection** — a book 45% Mystery + 35% Romance + 20% Fantasy gets properly classified as mixed
3. **Trend Analysis** — track which topics grow/shrink over time in reviews
4. **Personalized Recommendations** — match user's topic preferences to book topic distributions

Advantages:

- Unsupervised (no labeled data needed)
- Multi-topic documents handled naturally
- Interpretable (topics are word lists humans can read)

Limitations:

- K must be chosen before training
- Ignores word order (bag-of-words assumption)
- Works poorly on very short documents
- Topics may not always be coherent

(b) Word Cloud — Definition and Book Review Analysis [10 marks]

Definition: A word cloud (tag cloud) is a **visual representation of text** where word size reflects its frequency or importance. More frequent words appear larger and bolder.

Visual Properties:

- **Size** → frequency / importance
- **Color** → category or aesthetic (not analytically meaningful)
- **Position** → random / aesthetic (not meaningful)

How Word Clouds Work (6 Steps):

1. **Collect** — gather all text (book reviews)

2. **Preprocess** — lowercase, remove punctuation, remove stop words, tokenize
3. **Count** — frequency of each remaining word
4. **Map size** — `font_size ∝ (freq - min_freq) / (max_freq - min_freq)`
5. **Layout** — place words, avoid overlaps
6. **Render** — display with colors and font styles

Weight Option: Use **TF-IDF scores instead of raw frequency** for more meaningful word sizes (highlights distinctive words, not just common ones).

Simple Example:

Reviews combined text (after preprocessing):

```
excellent: 2 → LARGEST
plot: 2 → LARGE
characters: 2 → LARGE
writing: 2 → LARGE
amazing: 1, thrilling: 1, loved: 1 → medium
```

Visual result: EXCELLENT, PLOT, CHARACTERS, WRITING appear prominently.

How Word Clouds Help Analyze Book Reviews:

1. Quick Content Overview

- Instantly see what readers discuss most (plot, characters, writing)
- No need to read thousands of reviews

2. Genre Identification

```
DRAGON, MAGIC, WIZARD prominent → Fantasy
SPACE, ALIEN, TECHNOLOGY → Sci-Fi
LOVE, ROMANTIC, PASSION → Romance
```

3. Sentiment Assessment

```
Positive cloud: AMAZING, EXCELLENT, LOVED, BRILLIANT
Negative cloud: DISAPPOINTING, BORING, CONFUSING, WASTE
```

4. Aspect/Feature Extraction

- CHARACTERS (large) → readers focus on character development
- PLOT (large) → plot quality is key
- PACING (medium) → pacing mentioned frequently
- Insight: author should prioritize character and plot in next book

5. Comparative Analysis

- Generate separate clouds for Book 1 vs Book 2 by same author
- Book 1: PLOT, PACING, EXCITING dominant → plot-driven
- Book 2: CHARACTERS, EMOTIONAL, TOUCHING dominant → character-driven
- Use for targeted marketing to different reader segments

Bonus: Can generate separate word clouds for **positive vs negative** reviews to pinpoint what readers love vs hate.

Advantages:

- Instant visual insight — understand 10,000 reviews in seconds
- Accessible to non-technical stakeholders
- Fast to generate, low computational cost
- Flexible: can use TF-IDF weights, sentiment scores, or custom filters

Limitations:

Limitation	Example
No context	"not good" and "good" both show "good" as large — misleading
Oversimplification	Reduces nuanced opinions to bare word counts
Frequency ≠ Importance	Common generic words may dominate even after stop word removal

Best Practice: Use word clouds as a **first exploratory step**, then validate with quantitative methods like sentiment analysis or LDA for deeper insights.

Quick Reference Summary

Topic	Key Formula / Example	Location in Notes
TF-IDF calculation	He/Radhika corpus: IDF("good")=0, IDF("He")=0.301	Unit 1 notes
BoW	puppy sentence → frequency vector	Unit 1 notes
Word2Vec window	"The Sun rises in the east" — target "rises", context up to w=2	Unit 2 notes
CBOW vs Skip-gram	CBOW: context→target; Skip-gram: target→context	Unit 2 notes
FastText subword	awesome → awe, wes, eso, som, ome	Unit 2 notes
LDA example	D1-D5 food/animals, K=2 topics	Unit 2 notes
Cosine similarity	$\cos(\theta) = (A \cdot B)/(\|A\|\|B\|)$, range 0→1	Unit 2 notes

---

Good luck — you already know these examples from your notes!